

Caché de prompts

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 9 de 12 · Notas en español (es-ES)

Sin caché, cada llamada Messages es **sin estado**: el PDF, el system prompt enorme, el thinking sobre ese PDF — todo se tira tras la respuesta. El siguiente clic vuelve a pagar entrada (y thinking). La **caché de prompts** guarda un prefijo en lado Anthropic durante un TTL corto (en clase: **5 minutos o 1 hora**) para que las peticiones siguientes lo reutilicen a precio de acierto. Vijay habló también de **container id** para que los seguimientos se queden en la réplica que tiene la caché.

Por qué existe

El relato motivador es chatear con un PDF: convertir un documento de un millón de tokens y enviarlo en cada pregunta es una factura de entrada cada vez. Cachead el documento (y las instrucciones estables). Las preguntas de usuario siguientes son pequeñas; pagáis una escritura y luego aciertos. Combinar thinking extendido sin caché se describió como desperdicio doble: releer + repensar.

Cómo lo activó el laboratorio

En el contenido system (u otros bloques cacheables en las APIs actuales), poned `cache_control` con `type: ephemeral`. Esa bandera marca «cachead este prefijo». Ephemeral significa que el servidor solo recuerda durante la duración acordada y luego descarta — Vijay lo ligó a lenguaje de SLA: no es un almacén permanente ni vuestra base RAG.

El demo mezcló un «code prompt» / blob tipo HTML enorme como system (y se enredó un momento con tool calling cuando el notebook se torció). En concepto: poned lo **grande e invariante** detrás de `cache_control`; dejad la pregunta de cada turno después del punto de ruptura.

Write, hit, read — y dinero

Operación (clase)	Significado	Charla de precio en la diapositiva (ilustrativa)
Cache write	La primera vez que se guarda ese prefijo	Más cara que la entrada cruda; TTL de 5 minutos frente a 1 hora con precios distintos (p. ej. ~12,50 \$ frente a ~20 \$ / MTok en esa baraja)
Cache hit	Una petición posterior reutiliza el prefijo	Mucho más barata (del orden de ~1 \$ / MTok en esa baraja)
Cache read	Leer tokens cacheados al procesar	Puede ir incluido o facturarse según el modelo — mirad el precio vivo

Los dólares exactos del 17–20 de agosto de 2026 **no** son constantes de examen. La estructura es: escribir es caro, acertar es barato, la salida sigue siendo salida, los tokens de thinking existen si está activado. Ved las páginas vigentes de caché y precios.

Container id y enrutado pegajoso

Arquitectura de clase: contenedores LLM con escala horizontal. Una caché vive en la instancia que la escribió. La respuesta Message puede incluir un identificador de **container** (ya vislumbrado el 17 de agosto). Reenviadlo en la siguiente petición para caer en la misma caché. Si esparcís el tráfico por réplicas al azar, volvéis a pagar escrituras. Las docs oficiales pueden describir coincidencia automática de prefijos en vez de pin manual de contenedor — si discrepan del notebook, **ganan las docs**, pero estad listos para explicar el relato de réplicas de Vijay.

Qué no es la caché

- No es la memoria de sesión de Chat en sentido de producto — caduca.
- No es RAG / una base SQL que consultáis (preguntas del 17 de agosto).
- No sustituye reenviar el turno de usuario que *cambia*.
- No es el refresco de uso de cinco horas de Claude Pro.

Reglas de diseño

1. Prefijo estable y grande primero (system, tools, documentos).
2. Marcad cache_control en el punto de ruptura.
3. La pregunta variable, al final.
4. Reutilizad el mismo id de modelo; cambiar de modelo suele fallar la caché.
5. Mirad los campos usage: tokens de cache_creation frente a cache_read en la respuesta.

LOS MÍNIMOS DE TOKENS Y LOS NOMBRES DE TTL CAMBIAN

Las APIs de hoy documentan tamaños mínimos de prefijo y enums de TTL. En el aula la palabra era «ephemeral» más 5 minutos / 1 hora. Alinead con **prompt caching** antes del examen.

Puente a tools y MCP

El resto del 20 de agosto introdujo tools/MCP como vía para inyectar datos **en tiempo real y propietarios** — otro camino de mejora de contexto, examinado en el módulo 10. La caché abarata el contexto *estático*; las tools traen contexto *fresco*.

Documentación oficial

• Caché de prompts

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

cache_control, TTL, puntos de ruptura, campos usage.

• Messages API

<https://docs.claude.com/en/api/messages>

Dónde vive cache_control en system y en bloques de contenido.

- **Pensamiento extendido**

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

No volváis a pensar un PDF cacheado en cada turno sin necesidad.

- **Soporte PDF**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Los documentos son el prefijo grande habitual a cachear.

- **Límites de tasa**

<https://docs.claude.com/en/api/rate-limits>

El caudal sigue aplicando en los aciertos de caché.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.